


# How Photos Become 3D

Week 7 — Tuesday Lecture

## Today

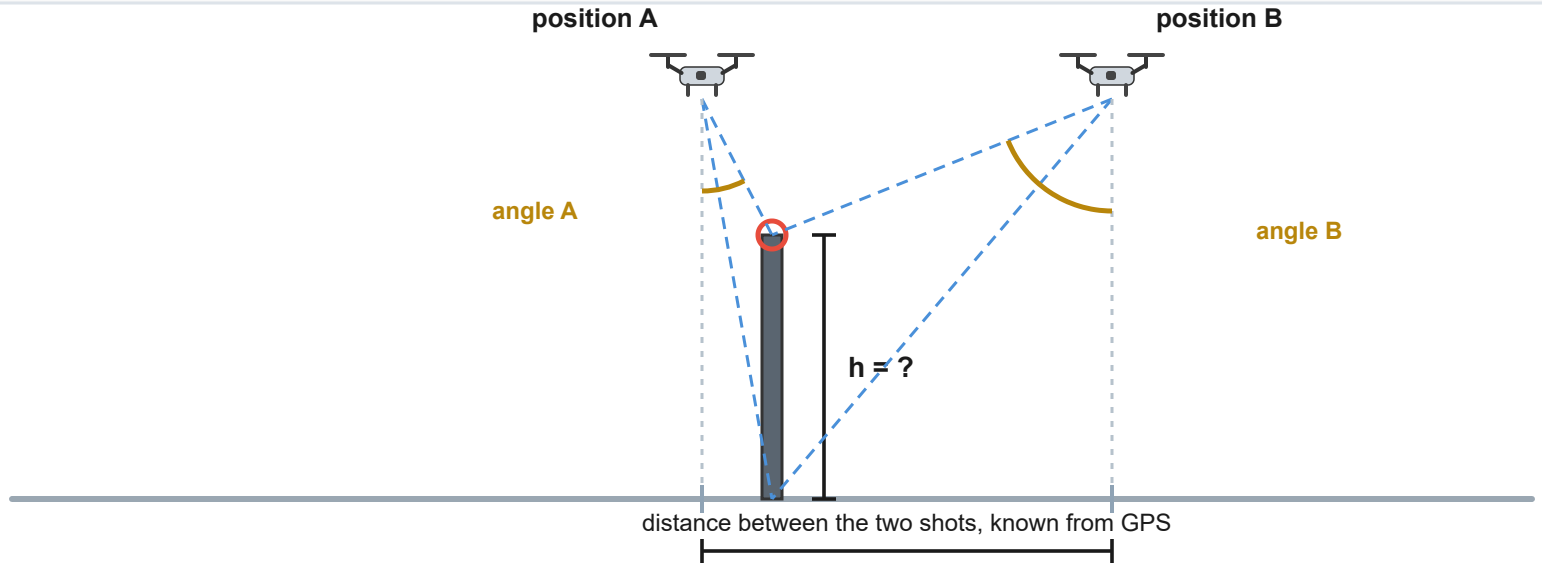
- Where 3D comes from: finding and matching points
- Where the cameras were
- From sparse to dense to mesh
- The products: orthophoto, DSM, DTM
- What the photo's metadata carries
- The software, briefly

# Two photos of the same corner of a building.

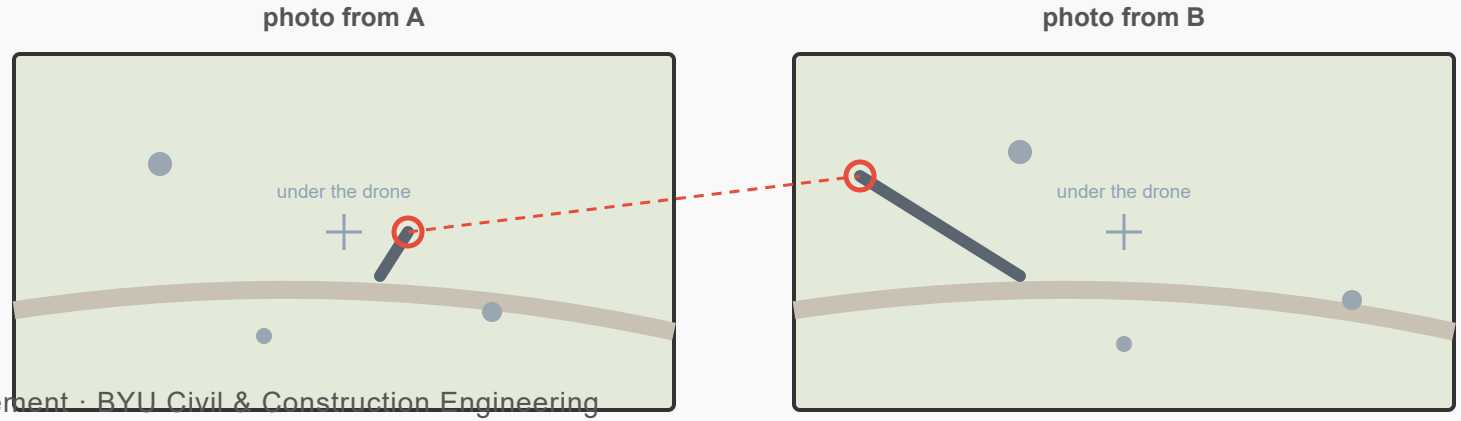
What does the computer actually know?

## Two photos, one pole, and where height comes from

The same point sits in a different place in each photo. That difference is what the software turns into 3D.




**Two known camera positions, two measured angles, one unknown height. That is a triangle, and trigonometry solves it.**




pole nearly underneath, so it looks short

pole off to the side, so it looks long

## The triangle idea

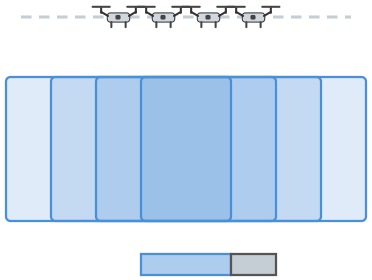
- Two photos of the same thing, taken from different places
- Two **known** camera positions
- Two **measured** angles
- One **unknown** height — solve the triangle

## Why overlap is the number that matters

Overlap exists so the same point lands in several photos. Without that, there is nothing to triangulate.

### Forward overlap, 80%

between one photo and the next



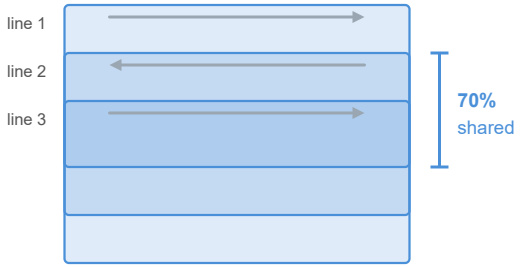
**80% of each photo repeats the one before**

only the grey 20% is new ground

so every point on the ground is caught by
5 to 9 different photos.

### Side overlap, 70%

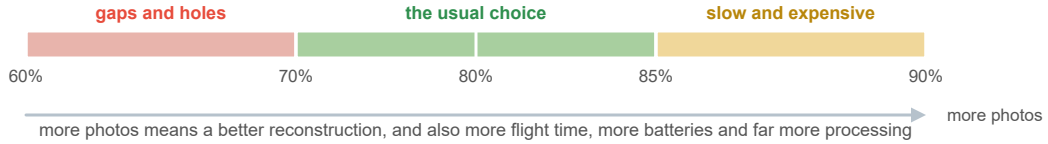
between one flight line and the next




**70% of each strip is photographed twice.**

Too little sidelap leaves gaps between lines
that no software can fill in afterwards.

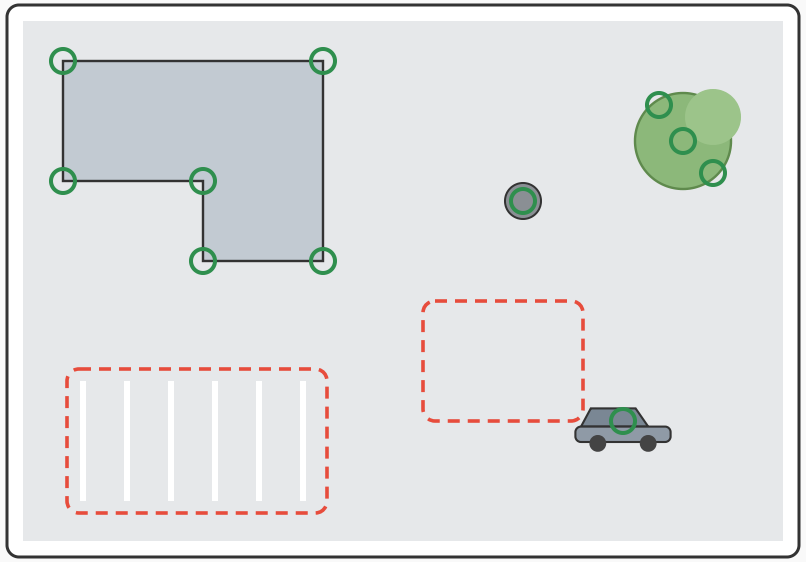
### What you are trading




Overlap is what puts every ground point into five to nine different photos.

## Where the software finds its points

A tie point has to be findable again in the next photo. Corners and texture are; blank and repeating surfaces are not.



**Keypoints: corners, edges that end, texture**

A building corner, a manhole lid, the edge of a car, the texture of a tree canopy. Each one looks like itself and like nothing nearby.

**Blank asphalt: nothing to grab**

Every pixel looks like every other pixel. The software finds no point here, and the model will be smooth and vague across it.

**Identical stripes: the wrong thing to grab**

Every stripe looks like every other stripe, so a point found here is matched to the wrong stripe in the next photo. Worse than blank.

Thousands of these per photo, found automatically. You never pick them. You fly so that they exist.

*Overcast light helps: no hard shadows that move between photos and look like features.*

The software looks for corners and texture. Blank asphalt gives it nothing; a row of identical stripes gives it the wrong thing.

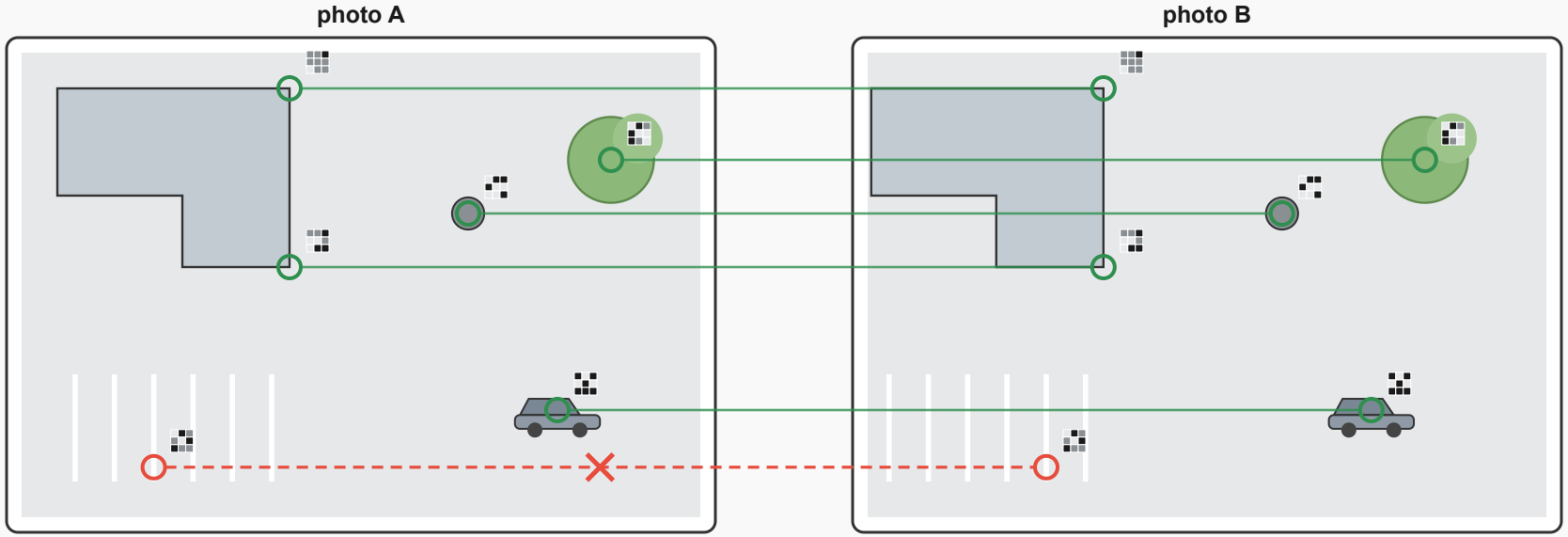
## What makes a good tie point

| Surface | Result |
|---|---|
| **Unique** — manhole, rock, corner, crack | Matched correctly |
| **Blank** — asphalt, snow, water, glass | Holes you can see |
| **Repeating** — stall lines, grass, row crops | Matched confidently and **wrongly** |

Overcast is good: flat light looks the same in every photo, and no moving shadows to fool the matcher.

## Matching: a fingerprint for every point

The pixels around a keypoint are summarised into a small pattern. Patterns that agree across two photos are a match.




**Green: fingerprints agree, geometry agrees**

The same corner, the same manhole, the same car, shifted the way everything else shifted between the two photos. Kept as tie points.

**Red: fingerprints agree, geometry does not**

Two identical stripes produce identical fingerprints. The match points the wrong way compared with every other match, so it is thrown out (RANSAC).

*A fingerprint is built so that it survives a change of scale and rotation: the next photo was taken from somewhere else.*

Each point gets a fingerprint of the pixels around it. Fingerprints that agree are matched; a match that disagrees with all the others is thrown out.

## SIFT, in plain words

- Finds **blobs and corners** that still stand out at several zoom levels
- Gives each one an **orientation**
- Summarises the pixels around it as **128 numbers** — the fingerprint
- Same corner, different photo: nearly the same numbers
- So a change of scale or rotation does not break the match

## Matching, then cleaning up

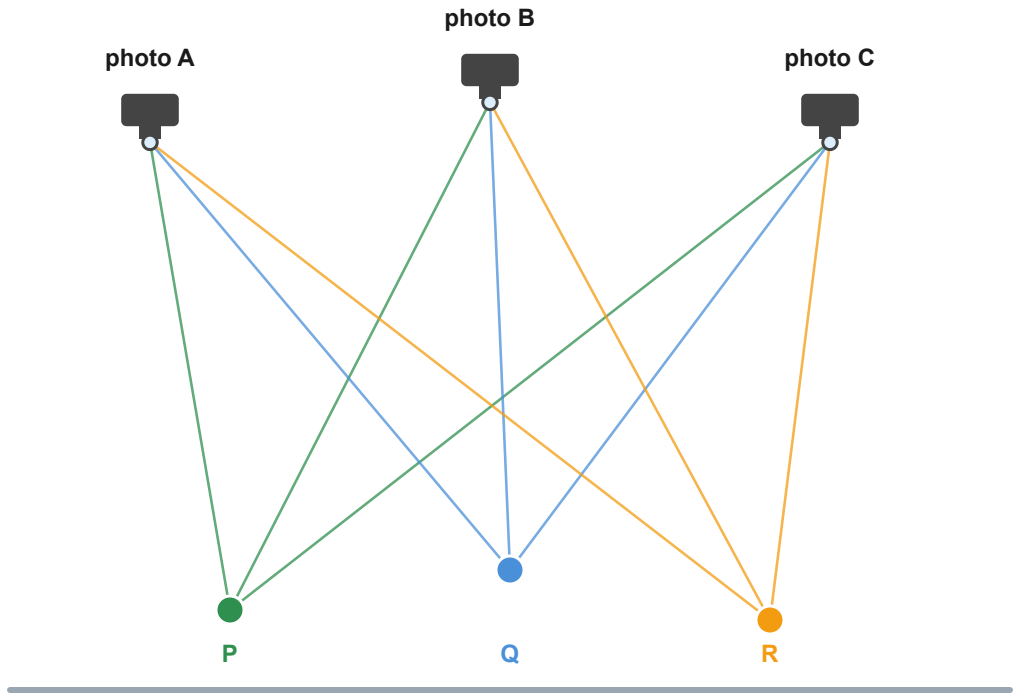
- Compare fingerprints; keep a match only if the best is **clearly better than the second best** — the ratio test
- **RANSAC**: guess a camera geometry from a few matches, count who agrees, repeat
- Matches that never agree with anything are **thrown out**
- Once the geometry is known, a partner point lies on **one line** in the other photo

# A parking lot of identical stripes.

Why is that worse than a blank field? Which test catches it — the ratio test, or RANSAC?

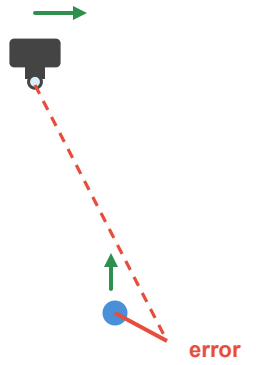
## Where was the camera? Where is the point? Solved together

Every matched point sends one ray from every photo that saw it. Move cameras and points until the rays agree.




three cameras, three points, nine rays: one solve

### A ray that misses




nudge the camera and the point until the miss is as small as it can be for every ray at once

**This is bundle adjustment.**

Bentley: aerotriangulation.

*The GPS tag on each photo is the first guess and sets the scale. The photos themselves do the rest.*

Cameras and points are solved together: nudge both until every ray through a matched point agrees. Bentley calls this aerotriangulation.

## Shape is not size

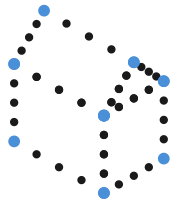
- GPS tags give the software a **first guess** and the **scale**
- With no known positions: right shape, wrong size, wrong place
- A building can come out perfectly proportioned and twice life size
- A reference, ground control, or **RTK** supplies the missing scale and position

# Think like the software

A rocky canyon and a snowfield, same day. Which one worries you? What do you change about the flight?

## From matched points to a model

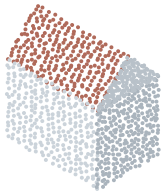
The same building at each stage. Only the amount of detail changes.



**Sparse cloud**

the tie points themselves

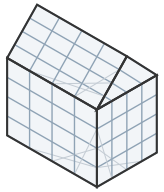
*hundreds of points*



**Dense cloud**

a point for almost every pixel

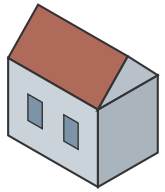
*millions of points*



**Mesh**

a surface stretched over the points

*triangles, not points*



**Textured model**

the photos draped back on

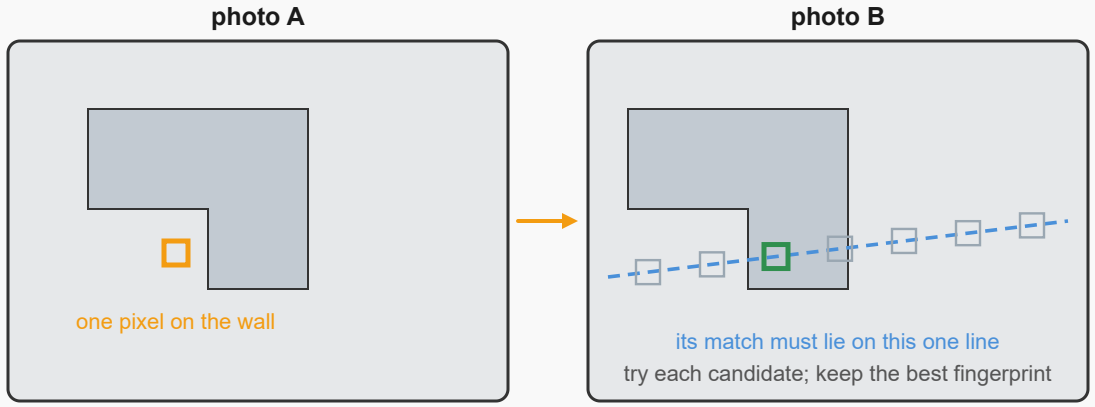
*what you show a client*

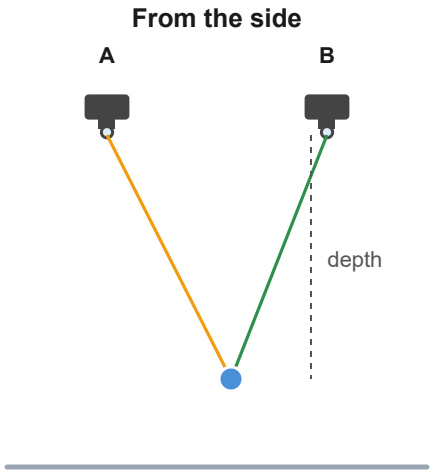
*Every stage after the first is derived. Get the tie points wrong and nothing downstream can recover.*

The same building at four stages: sparse, dense, mesh, textured.

## From thousands of points to millions

Once the cameras are known, every pixel can be matched, not just the distinctive ones, because the search is one line long.




**Repeat for every pixel of every photo**

Thousands of tie points become millions of surface points. Blank asphalt still gives nothing: no fingerprint, no match.

This is the slow step. It is why a reconstruction runs for hours while aerotriangulation took minutes.

*Multi-view stereo, in the appendix. Bentley: Reconstruction.*

For one pixel in photo A, the match in photo B lies on one known line. Find it, read the depth, repeat for every pixel.

## Sparse to dense

- Thousands of points become **millions**
- Once cameras are known, the search is along one line, not the whole image
- Needs **texture**, **overlap**, and **sharp** photos

## Points become triangles

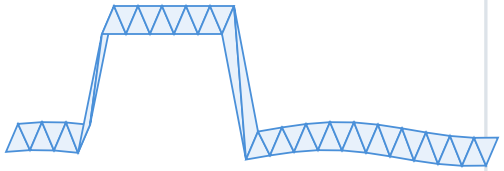
A cloud is not a surface. Joining neighbouring points into triangles makes one, and the one rule that matters is how far apart two points may be and still be joined.

### Dense points



a side slice through the cloud

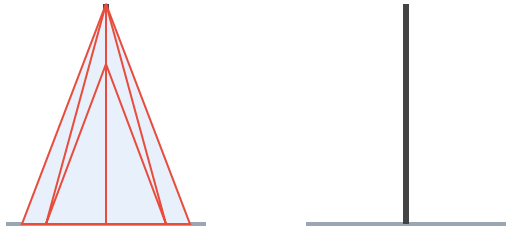
### Joined into triangles



every point joined to its neighbours

### The thin pole problem

a pole or a wire has points far from the ground points beside it; join them and you get a tent




*A webbed mesh is not a bad flight. It is a tolerance to change in the software.*

Points become triangles. A thin pole gets webbed to the ground unless the tolerance is tightened.

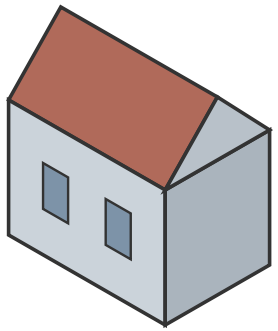
## Mesh and textured model

- **Mesh:** a skin of triangles over the points; thin things like railings and wires often vanish
- **Texture:** each triangle colored from the photo that saw it best — recognizable, not more accurate
- Mesh looks webbed? **Tighten the tolerance.** More honest gaps, fewer invented surfaces

## From the surface to the deliverables

The orthophoto and the DSM are the same surface, read two different ways.

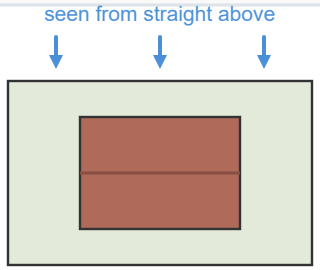
### The surface



meshed from the dense cloud,
with the photos draped over it

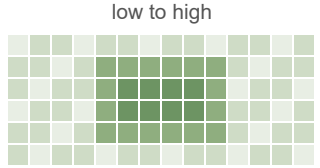
### Orthophoto

every pixel put where it truly belongs,
at one constant scale across the whole image




### Surface model (DSM)

the height of that surface, sampled onto
a regular grid, one height per cell



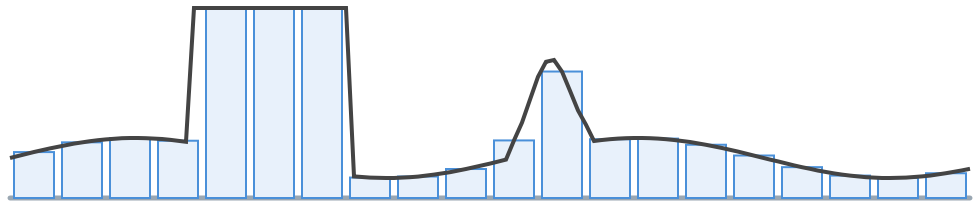

*Strip the buildings and vegetation out of that grid and the same surface becomes a DTM.*

The orthophoto and the DSM are the same surface read two ways: a picture, and a grid of heights.

## A DEM is the surface sampled on a grid

Lay a grid over the mesh and ask one question in every cell: how high is the surface here? The answer is one number per cell.

### From the side



the mesh (black) and the height recorded in each cell (blue)

### From above: the grid of heights

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 21 | 24 | 27 | 27 | 28 | 27 | 25 | 24 |
| 24 | 27 | 26 | 27 | 26 | 26 | 24 | 20 |
| 27 | 95 | 95 | 95 | 27 | 24 | 21 | 18 |
| 26 | 95 | 95 | 95 | 24 | 20 | 18 | 16 |
| 26 | 95 | 95 | 95 | 21 | 19 | 18 | 16 |
| 27 | 26 | 24 | 21 | 18 | 18 | 62 | 17 |
| 27 | 24 | 21 | 18 | 16 | 15 | 16 | 16 |
| 25 | 21 | 20 | 18 | 14 | 15 | 17 | 18 |

one height per cell, the cell is the GSD

*Keep the building and the tree and it is a DSM. Strip them to bare ground and it is a DTM.*

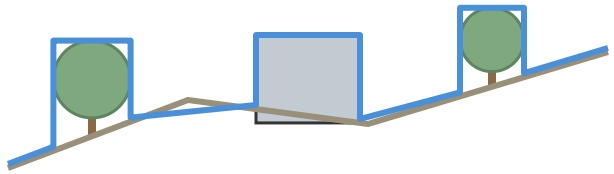
Sample the surface on a grid and every cell gets a height. Keep the trees and cars for a DSM; strip them for a DTM.

## What the model includes

Same flight, same site, two different surfaces. Which one you need depends on the question.

### Surface model (DSM)

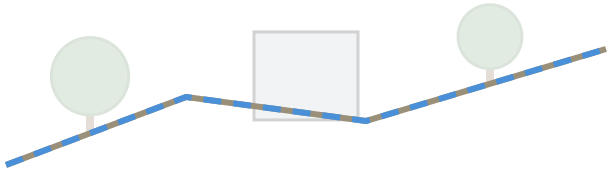
everything the drone saw



use it to check clearances and heights

### Terrain model (DTM)

bare ground, trees and buildings stripped out



use it for earthwork, drainage and grading

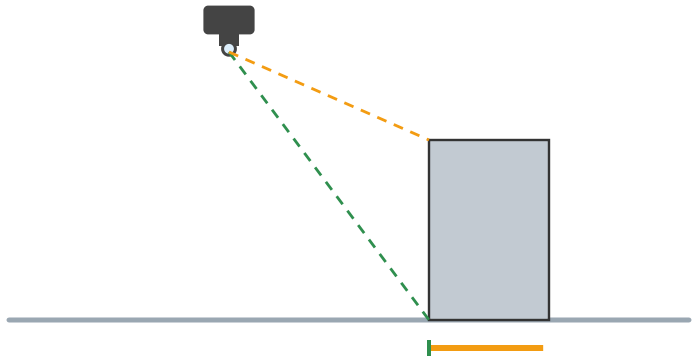
*Digital Elevation Model (DEM) is the umbrella term you will also hear for both of these.*

DSM keeps the trees and buildings. DTM strips them out — only one is right for earthwork.

## Why a photo is not a map, and how the mesh fixes it

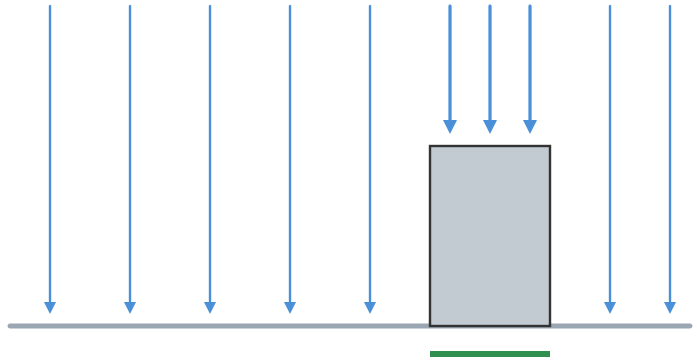
A camera sees the top of a building further from the centre than its base. Projecting every photo onto the mesh and looking straight down removes that lean.

### The photo




buildings lean away from the centre of every photo
and two photos disagree about where the roof is

### The orthophoto




every point viewed from straight above, using the
mesh to know where each pixel sits on the ground

*Orthorectification. Stitch the straightened photos together and the result is the orthomosaic: a picture you can measure on.*

*Lean that survives into the orthomosaic means the mesh was wrong there, usually from too little overlap.*

A photo leans every building away from the camera. Drape each photo on the mesh and look straight down at every point, and the lean is gone.

# Which product answers which question?

"How much gravel is in that pile?" and "Where does the curb run?"

## Metadata: data about data

- The **can of soup**: the soup is the data, the label is the metadata
- A photo of a site is not much use without its context
- Drones depend on it: **where, when, which way**
- Bad metadata can make good data useless

## What EXIF carries

- **Position:** latitude and longitude
- **Altitude:** sets the scale of the image
- **Time:** the timestamp, for change over time
- **Camera and lens settings:** ISO, shutter speed
- **Orientation:** heading (yaw) and gimbal pitch, nadir or oblique

Stored inside the image file. Right-click, Properties, Details to see it.

# One real photo's EXIF



Parrot Anafi, Utah Lake shore, May 2023

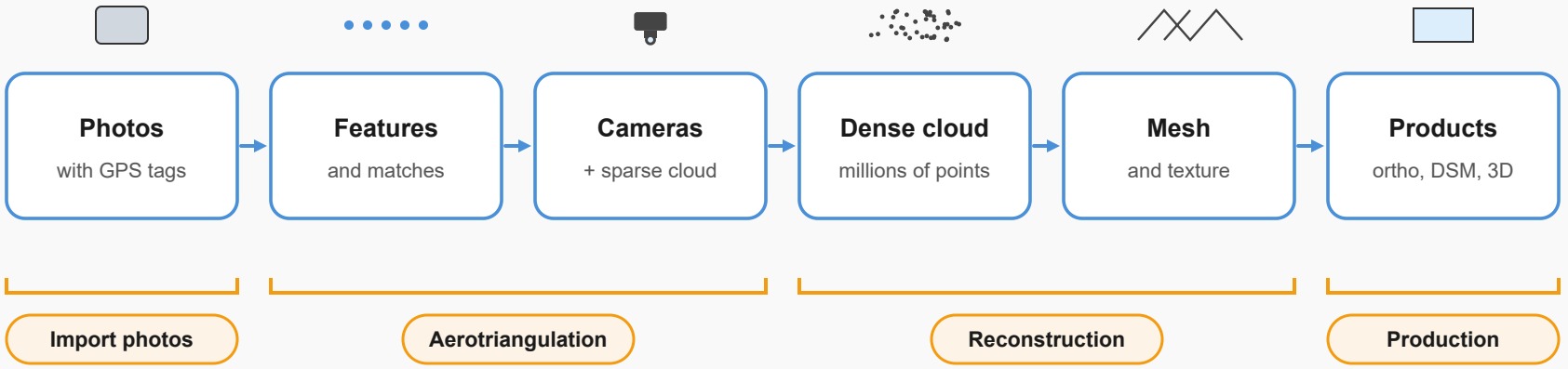
| Field | Value |
|---|---|
| Position | 40° 20′ 12.9″ N, 111° 46′ 39.5″ W |
| Altitude | 1417.2 m above sea level |
| Time | 2023-05-26 13:14:08 |
| Image | 4608 × 3456 px |
| Lens | 4.0 mm, f/2.4 |
| Exposure | 1/3846 s, ISO 142 |
| Camera pitch | 6.4° — nearly nadir |

## Metadata and privacy

- Exact GPS coordinates are in every photo — and this one also carried the aircraft **serial number** and firmware
- Social media strips them on upload to protect you
- For engineering we **must keep** them intact
- Check the drone's clock and GPS lock before you fly

## Six stages, four buttons

What structure from motion does, and what the software calls each part of it.

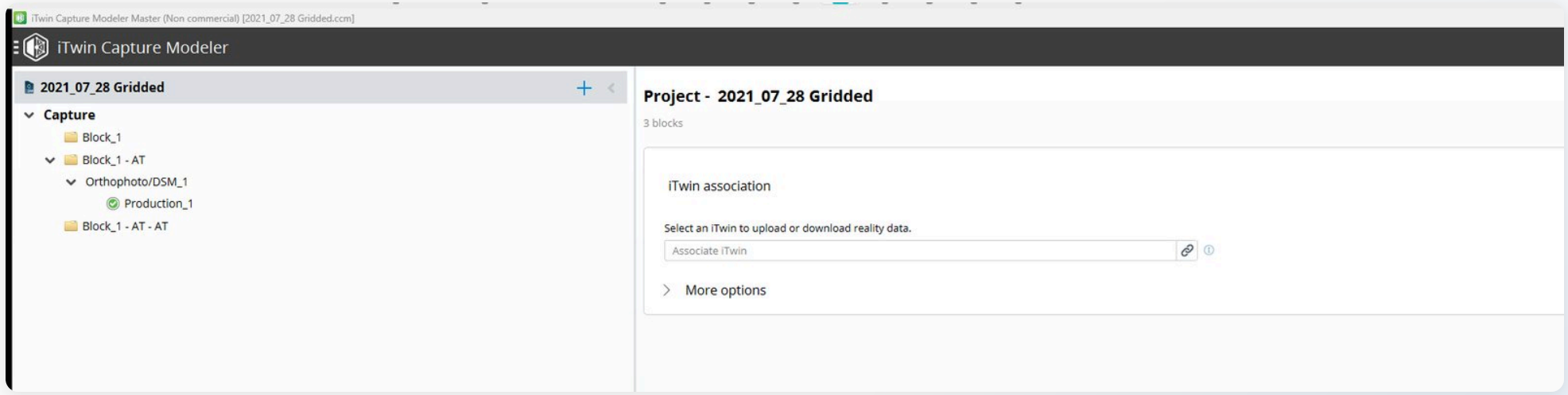



Bentley iTwin Capture Modeler, the four steps you click

*The software does the arithmetic. The photos decide whether the arithmetic has anything to work with.*

Six stages, four buttons. Every product on the right is built from the photos on the left.

# Bentley iTwin Capture Modeler




One project: a block of photos, its aerotriangulated copy, a reconstruction, and a production. iTwin Capture Modeler, Bentley Systems.

## Create your Bentley account

Before the processing lab. Learning Suite has the link.

- Lab deck: Bentley iTwin Capture Modeler
- Bentley iTwin

## What to do Thursday

- **Rock Canyon flight day:** Rock Canyon Park Flight
- Fly the plan from the Week 6 lab
- Bring back the photos — they become next week's model
- Create your **Bentley account** before the processing lab